1) Hill climbing

```
import random


def calculate_cost(board):
    """Count pairs of queens attacking each other."""
    n = len(board)
    cost = 0

    for i in range(n):
        for j in range(i + 1, n):
            # Same row or same diagonal
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                cost += 1

    return cost


def hill_climbing(n=4, initial_board=None):
    while True:
        if initial_board is not None:
            current = initial_board[:]
            initial_board = None
        else:
            current = [random.randrange(n) for _ in range(n)]

        print("\nStarting state:", current)

        while True:
            current_cost = calculate_cost(current)
            print("\nCurrent state:", current)
            print("Current cost:", current_cost)

            if current_cost == 0:
                print("Goal state reached!")
                return current

            best_neighbor = None
            best_cost = current_cost

            # Generate all neighbors by moving one queen
            for col in range(n):
```

```
            for row in range(n):
                if row == current[col]:
                    continue

                neighbor = current[:]
                neighbor[col] = row
                neighbor_cost = calculate_cost(neighbor)

                print(
                    "Neighbor:", neighbor,
                    "Cost:", neighbor_cost
                )

                if neighbor_cost < best_cost:
                    best_cost = neighbor_cost
                    best_neighbor = neighbor

        if best_neighbor is None:
            print("Local minimum reached. Restarting...")
            break

        current = best_neighbor
        print("Selected neighbor:", current)
        print("Selected cost:", best_cost)


def display_board(board):
    n = len(board)

    for row in range(n):
        for col in range(n):
            if board[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()


# Example initial configuration; replace with the PPT's board.
initial_board = [0, 1, 2, 3]

solution = hill_climbing(4, initial_board)

print("\nFinal solution:", solution)
```

```
print("Final cost:", calculate_cost(solution))
display_board(solution)
```

IDLE Shell 3.12.3

File Edit Shell Debug Options Window Help

```
Starting state: [0, 1, 2, 3]

Current state: [0, 1, 2, 3]
Current cost: 6
Neighbor: [1, 1, 2, 3] Cost: 4
Neighbor: [2, 1, 2, 3] Cost: 5
Neighbor: [3, 1, 2, 3] Cost: 4
Neighbor: [0, 0, 2, 3] Cost: 4
Neighbor: [0, 2, 2, 3] Cost: 4
Neighbor: [0, 3, 2, 3] Cost: 5
Neighbor: [0, 1, 0, 3] Cost: 5
Neighbor: [0, 1, 1, 3] Cost: 4
Neighbor: [0, 1, 3, 3] Cost: 4
Neighbor: [0, 1, 2, 0] Cost: 4
Neighbor: [0, 1, 2, 1] Cost: 5
Neighbor: [0, 1, 2, 2] Cost: 4
Selected neighbor: [1, 1, 2, 3]
Selected cost: 4

Current state: [1, 1, 2, 3]
Current cost: 4
Neighbor: [0, 1, 2, 3] Cost: 6
Neighbor: [2, 1, 2, 3] Cost: 5
Neighbor: [3, 1, 2, 3] Cost: 4
Neighbor: [1, 0, 2, 3] Cost: 2
Neighbor: [1, 2, 2, 3] Cost: 3
Neighbor: [1, 3, 2, 3] Cost: 3
Neighbor: [1, 1, 0, 3] Cost: 3
Neighbor: [1, 1, 1, 3] Cost: 4
Neighbor: [1, 1, 3, 3] Cost: 4
Neighbor: [1, 1, 2, 0] Cost: 2
Neighbor: [1, 1, 2, 1] Cost: 5
Neighbor: [1, 1, 2, 2] Cost: 3
Selected neighbor: [1, 0, 2, 3]
Selected cost: 2

Current state: [1, 0, 2, 3]
Current cost: 2
Neighbor: [0, 0, 2, 3] Cost: 4
Neighbor: [2, 0, 2, 3] Cost: 2
Neighbor: [3, 0, 2, 3] Cost: 2
Neighbor: [1, 1, 2, 3] Cost: 4
Neighbor: [1, 2, 2, 3] Cost: 3
Neighbor: [1, 3, 2, 3] Cost: 3
Neighbor: [1, 0, 0, 3] Cost: 2
Neighbor: [1, 0, 1, 3] Cost: 3
Neighbor: [1, 0, 3, 3] Cost: 3
Neighbor: [1, 0, 2, 0] Cost: 2
Neighbor: [1, 0, 2, 1] Cost: 3
Neighbor: [1, 0, 2, 2] Cost: 3
Local minimum reached. Restarting...

Starting state: [1, 0, 3, 1]

Current state: [1, 0, 3, 1]
Current cost: 3
Neighbor: [0, 0, 3, 1] Cost: 1
Neighbor: [2, 0, 3, 1] Cost: 0
Neighbor: [3, 0, 3, 1] Cost: 1
Neighbor: [1, 1, 3, 1] Cost: 4
```

```
Current state: [1, 0, 2, 3]
Current cost: 2
Neighbor: [0, 0, 2, 3] Cost: 4
Neighbor: [2, 0, 2, 3] Cost: 2
Neighbor: [3, 0, 2, 3] Cost: 2
Neighbor: [1, 1, 2, 3] Cost: 4
Neighbor: [1, 2, 2, 3] Cost: 3
Neighbor: [1, 3, 2, 3] Cost: 3
Neighbor: [1, 0, 0, 3] Cost: 2
Neighbor: [1, 0, 1, 3] Cost: 3
Neighbor: [1, 0, 3, 3] Cost: 3
Neighbor: [1, 0, 2, 0] Cost: 2
Neighbor: [1, 0, 2, 1] Cost: 3
Neighbor: [1, 0, 2, 2] Cost: 3
Local minimum reached. Restarting...

Starting state: [1, 0, 3, 1]

Current state: [1, 0, 3, 1]
Current cost: 3
Neighbor: [0, 0, 3, 1] Cost: 1
Neighbor: [2, 0, 3, 1] Cost: 0
Neighbor: [3, 0, 3, 1] Cost: 1
Neighbor: [1, 1, 3, 1] Cost: 4
Neighbor: [1, 2, 3, 1] Cost: 4
Neighbor: [1, 3, 3, 1] Cost: 4
Neighbor: [1, 0, 0, 1] Cost: 4
Neighbor: [1, 0, 1, 1] Cost: 5
Neighbor: [1, 0, 2, 1] Cost: 3
Neighbor: [1, 0, 3, 0] Cost: 3
Neighbor: [1, 0, 3, 2] Cost: 4
Neighbor: [1, 0, 3, 3] Cost: 3
Selected neighbor: [2, 0, 3, 1]
Selected cost: 0

Current state: [2, 0, 3, 1]
Current cost: 0
Goal state reached!

Final solution: [2, 0, 3, 1]
Final cost: 0
. Q . .
. . . Q
Q . . .
. . Q .
>>>
```